

API Documentation

1. Introduction

The goal of this API is to allow clients to consume data efficiently and to provide an LLM assistant that can help a user explore and understand the data in natural language.

The system is built using Python, with FastAPI handling the backend, Apache Spark managing the analytical workloads, and Streamlit providing the user interface for the Agentic AI.

2. RESTFul API implementation

To present the financial data stored in MongoDB, it was designed a RESTful API using the FastAPI framework.

The API endpoints are linked to the functional requirements of the project. For instance, the system provides endpoints to retrieve limited information about all financial assets, get specific details using an identifier, and fetch time-series data based on specific queries.

```
@main.get("/assets")
def get_assets(skip: int = 0, limit: int = 100):
    assets = AssetRepository.get_all(skip=skip, limit=limit)
    result = [{"id": a["_id"], "symbol": a["symbol"]} for a in assets]
    return {"status": "success", "count": len(result), "data": result}
```

(Fig. 1 “API handling the retrieval of assets”)

2.1. Analytics and data processing

Beyond the simple operations, the API serves as a bridge to the analytics engine. It was integrated Apache Spark to compute aggregations (minimum, maximum, mean) and to run forecasting models directly through the API endpoints.

For example, when a user requests a statistical summary, the API calls the Spark analytics workflow as shown in the figure 2:

```
@main.get("/analytics/summary")
def get_analytics_summary(asset_id: str):
    asset = AssetRepository.get_by_id(asset_id)
    if not asset:
        raise HTTPException(status_code=404, detail="Asset not found in the database")

    symbol = str(asset.get("symbol", "")).upper()

    try:
        stats = SparkAnalyticsWorkflow.run_aggregation_pipeline(asset_id, symbol)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Critical Spark Error: {str(e)}")

    if not stats:
        raise HTTPException(status_code=404, detail=f"No time series data found for {symbol} to compute summary")
```

(Fig 2 “Available get endpoints for data retrieval and analytics”)

3. Agentic AI and MCP integration

In order to create the LLM assistant, it was implemented a chat interface using Streamlit and the Google Gemini API (model gemini-2.0-flash-lite).

Instead of relying on generic finance text, the assistant is locked in the platform's actual data. To achieve this, it was used the Model Context Protocol (MCP) so the LLM can call the platform's data as tools.

The tools are defined by making HTTP calls to the FastAPI backend. If the LLM needs to know the available assets or compute a forecast, it triggers the corresponding Python function as presented in the figure 3:

```
def get_asset_forecast(asset_id: str):
    response = requests.get(f"{base_url}/analytics/forecast", params={'asset_id': asset_id})
    return response.text

#Initialize the chat session
if "chat_session" not in st.session_state:
    st.session_state.chat_session = client.chats.create(
        model="models/gemini-2.0-flash-lite",
        config=types.GenerateContentConfig(
            tools=[get_assets, get_asset_statistics, get_asset_forecast, get_vendors]
        )
    )
```

(Fig 3 “AI initialization and tool calling configuration”)

By providing these functions as tools to the Gemini configuration, the system achieves an agent-style workflow. When a user asks "What is the forecast for MSFT?", the LLM recognizes that it lacks this information, pauses its text generation, executes the `get_asset_forecast` tool to fetch the Spark prediction from the database, and then formulates a natural language response based on the actual data returned.

4. Conclusion

The resulting platform successfully combines a temporal NoSQL storage layer with a fast RESTful API. By extending the architecture with an Agentic AI interface, the system not only stores heterogeneous financial data but also makes it highly accessible for non-technical users to explore through natural language prompts.